PROJECT: BUILD A GOOGLE MEET–LIKE APP (MERN STACK)

Objective

Students should build a **real-time video conferencing application** similar to Google Meet with essential and optional features.

🧩 PHASE 1: Problem Understanding (Analysis First)

Answer these before coding:

🔍 Core Questions

- What is a video conferencing system?
- What are the **minimum features** required?
- Who are the users? (host, participant)
- What happens when:
    - A user joins?
    - A user leaves?
    - Internet disconnects?

🧠 Expected Thinking

They should identify:

- Real-time communication is needed
- Video + audio streaming required
- Users need identification (name/email)

📐 PHASE 2: Feature Breakdown

✅ Must-Have Features (MVP)

Students should implement:

- User can **create a meeting**
- User can **join via meeting ID/link**

- **Video + Audio streaming**
- **Mute / Unmute**
- **Turn camera on/off**
- Show **participants list**

🚀 **Intermediate Features**

- Chat system (real-time)
- Screen sharing
- Raise hand feature
- Notifications (user joined/left)

🔥 **Advanced Features (Optional)**

- Recording meetings
- Authentication (JWT)
- Waiting room
- Admin controls (kick user)
- Background blur

🧱 **PHASE 3: System Design Thinking**

Students should design BEFORE coding:

🧭 **Architecture Questions**

- How will frontend talk to backend?
- How will users connect in real-time?

💡 **Key Concepts to Discover**

- WebRTC (for video/audio)
- Socket connections (real-time events)

- REST APIs (basic data)

⚙️ PHASE 4: Tech Stack Mapping

Explain how MERN fits:

🟢 Frontend (React)

- UI for video grid
- Controls (mute, camera)
- Routing (join/create meeting)

🟡 Backend (Node + Express)

- Meeting management
- User sessions
- API endpoints

🔵 Database (MongoDB)

- Store:
    - Users
    - Meetings
    - Chat history

🔴 Real-Time Layer

- WebRTC → video/audio
- Socket.IO → signaling + chat

🔄 PHASE 5: Step-by-Step Development Plan

🪜 Step 1: Basic Setup

- Create MERN project
- Setup Express server

- Connect MongoDB

🪜 Step 2: Meeting System

- Create meeting ID
- Join meeting route
- Simple UI (no video yet)

🪜 Step 3: Real-Time Communication

- Setup Socket.IO
- Notify when users join/leave

🪜 Step 4: Video Calling (Core)

- Implement WebRTC:
    - Peer connection
    - Media streams
- Show video grid

🪜 Step 5: Controls

- Mute/unmute
- Camera toggle

🪜 Step 6: Chat Feature

- Real-time messaging
- Show messages in UI

🪜 Step 7: Polish UI

- Responsive layout
- Better UX

🧪 PHASE 6: Testing & Edge Cases

Students should think about:

- What if user refreshes page?
- What if network is slow?
- What if camera permission denied?

📊 Evaluation Criteria (Give them this!)

Area	Marks
Functionality	30%
Code Structure	20%
UI/UX	15%
Real-time handling	20%
Creativity	15%

🧠 Important Learning Outcomes

Yo will learn:

- Real-time systems design
- WebRTC basics
- MERN architecture
- Debugging async systems

💡 Guidance for you (Very Important)

👉 Don’t try to build everything at once
👉 First make it **work for 2 users only**
👉 Then scale features step by step

🗣️ Final Instruction

“Don’t copy Google Meet. Understand how it works, break it into parts, and rebuild it step by step.”